



JECRCTM
UNIVERSITY
BUILD YOUR WORLD

Computer Programming in C++

Unit-4

Object-Oriented Programming Concepts

Constant & Static Keywords



Constant Keyword:

- Used to define constant variables whose values cannot be changed after initialization.
- Example:
 - `const int x = 10;`

Static Keyword:

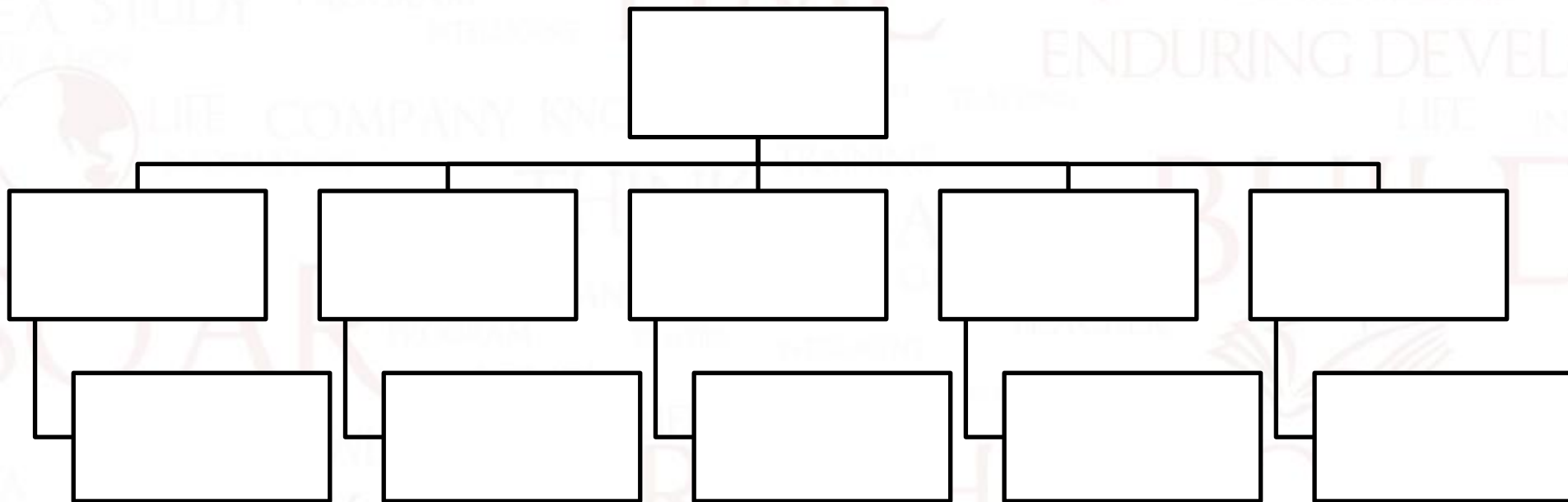
- Used with variables and functions to define class-wide (or function-wide) properties.
- **Static variables** inside a function retain their value between function calls.
- **Static members in a class** are shared among all objects of the class.
- Example:
 - `class Test {`
 - `static int count;`
 - `};`



Inheritance



- Inheritance allows a class (**child class**) to inherit properties and behaviors from another class (**parent class**). This promotes **code reusability**.



Single Inheritance



□ One class inherits from another

```
#include <iostream>
using namespace std;

class Parent {
public:
    void show() { cout << "Single Inheritance\n"; }
};

class Child : public Parent {}; // Child inherits Parent

int main() {
    Child obj;
    obj.show(); // Inherited function
}
```



Multiple Inheritance



□ A class inherits from multiple base classes

```
#include <iostream>
using namespace std;
```

```
class A { public: void showA() { cout << "Class A\n"; } };
class B { public: void showB() { cout << "Class B\n"; } };
```

```
class C : public A, public B {}; // Inheriting from A and B
```

```
int main() {
    C obj;
    obj.showA();
    obj.showB();
}
```



Multilevel Inheritance



□ A class inherits from a derived class

```
#include <iostream>
using namespace std;
```

```
class A { public: void showA() { cout << "Class A\n"; } };
class B : public A { public: void showB() { cout << "Class B\n"; } };
class C : public B { public: void showC() { cout << "Class C\n"; } };
```

```
int main() {
    C obj;
    obj.showA();
    obj.showB();
    obj.showC();
}
```



Hierarchical Inheritance



□ Multiple classes inherit from a single base class

```
#include <iostream>
using namespace std;
class Parent { public: void show() { cout << "Parent Class\n"; } };

class Child1 : public Parent {};
class Child2 : public Parent {};

int main() {
    Child1 obj1;
    Child2 obj2;
    obj1.show();
    obj2.show();
}
```



Hybrid Inheritance



□ Combining Multilevel and Multiple Inheritance

```
#include <iostream>
using namespace std;
class A { public: void showA() { cout << "Class A\n"; } };
class B : public A { public: void showB() { cout << "Class B\n"; } };
class C { public: void showC() { cout << "Class C\n"; } };

class D : public B, public C {}; // Inheriting from B and C

int main() {
    D obj;
    obj.showA();
    obj.showB();
    obj.showC();
}
```



Constructor and Destructor in Inheritance



Constructor in inheritance:

- When an object of a derived class is created, the **base class constructor is called first**, followed by the derived class constructor.

Destructor in inheritance:

- Destructors work in the **reverse order of constructors**.
- The derived class destructor is called first, then the base class destructor.

Example :

```
#include <iostream>
using namespace std;
```

```
class Base {
public:
    Base() { cout << "Base Constructor\n"; }
    ~Base() { cout << "Base Destructor\n"; }
};
```

```
class Derived : public Base {
public:
    Derived() { cout << "Derived
Constructor\n"; }
    ~Derived() { cout << "Derived
Destructor\n"; }
};
```

```
int main() {
    Derived obj;
    return 0;
}
```



Function Overriding



- Function overriding occurs when a **derived class provides a specific implementation of a function** that is already defined in its base class.
- The function signature (name and parameters) must be the same.
- The **base class function must be declared as virtual for proper**

□ Example:

```
class Base {  
public:  
    virtual void show() { cout << "Base class function\n"; }  
};  
  
class Derived : public Base {  
public:  
    void show() override { cout << "Derived class function\n"; }  
};  
  
int main() {  
    Base* obj = new Derived();  
    obj->show(); // Calls Derived class function (because of `virtual`)  
}
```



Virtual Function



- A **virtual function** allows **runtime polymorphism** by ensuring that the function call is resolved dynamically (at runtime).
- Declared using the **virtual** keyword in the base class.
- Helps in **function overriding**.

□ Example:

```
class Base {  
public:  
    virtual void display() { cout << "Base class\n"; }  
  
class Derived : public Base {  
public:  
    void display() override { cout << "Derived class\n"; }  
  
int main() {  
    Base* ptr;  
    Derived d;  
    ptr = &d;  
    ptr->display(); // Calls Derived class's display() due to virtual function  
}
```



Pure Virtual Function



- A **pure virtual function** is a virtual function that **must be overridden** in the derived class.
- Declared using `= 0` in the base class.
- A class with at least one pure virtual function becomes an **abstract class**.

□ Example:

```
class Shape {  
public:  
    virtual void draw() = 0; // Pure virtual function  
};  
  
class Circle : public Shape {  
public:  
    void draw() override { cout << "Drawing Circle\n"; }  
};
```



Abstract Class



- ❑ A **class** that contains at least one pure virtual function.
- ❑ Cannot be instantiated directly.
- ❑ Serves as a **base class** for other classes.
- ❑ **Example:**

```
class Animal {  
public:  
    virtual void makeSound() = 0; // Abstract method  
};
```

```
class Dog : public Animal {  
public:  
    void makeSound() override { cout << "Woof!\n"; }  
};
```

- ❑ **Key Point** : You cannot create an object of an abstract class:

- ❑ `Animal a;` // ❌ Not allowed
- ❑ `Animal* a = new Dog();` // ✅ Allowed



Virtual Base Class



- **Used in multiple inheritance** to avoid duplication of members from a common base class.
- **Avoids the "Diamond Problem"** in multiple inheritance.

□ Example:

```
class A {  
    public:  
        int x;  
};
```

```
class B : virtual public A {}; // Virtual inheritance
```

```
class C : virtual public A {}; // Virtual inheritance
```

```
class D : public B, public C {}; // Avoids duplicate 'x'
```

- **Without virtual** , class D would inherit x twice (once from B, once from C)

